SR UNIVERSITY

SCHOOL OF COMPUTER SCIENCE & ARTIFICIAL INTELLIGENCE

COURSE PROJECT REPORT

SMART CAMPUS LOST-AND-FOUND MANAGEMENT SYSTEM

Submitted in partial fulfilment of the requirements for the course

DEVOPS AND FULL STACK DEVELOPMENT

Student Details	Information
Student HT No.	[Enter Hall Ticket / Enrolment Number]
Student Name	[Enter Student Full Name]
Faculty Coordinator	Dr. Mohammed Ali Shaik, Professor, School of Computer Science & Artificial Intelligence
Designation	Professor
School	School of Computer Science & Artificial Intelligence
Academic Year	2026–2027
Semester	I
Course Code	23CS102PE405

SR UNIVERSITY

Ananthasagar, Hasanparthy, Warangal, Telangana, India – 506371

www.sru.edu.in

TABLE OF CONTENTS

Chapter	Title	Suggested Contents	Page No.
1	Introduction	Background, problem statement, objectives, scope and limitations	
2	Requirement Analysis	Stakeholders, functional requirements, non-functional requirements and feasibility	
3	System Design	Architecture, use cases, data flow, database design and UI wireframes	
4	Implementation	Technology stack, modules, APIs, validation and security	
5	Testing and Results	Test plan, test cases, results, screenshots and defect fixes	
6	DevOps and Deployment	Git workflow, CI/CD, Docker, Kubernetes, monitoring and deployment	
7	Conclusion and Future Scope	Outcomes, limitations and future enhancements	
—	References and Appendices	References, user guide, API list and additional evidence	

ABSTRACT

[Write 150–250 words covering the campus lost-property problem, proposed web-based solution, main user roles, key features, technology stack, database, and DevOps deployment approach. Mention the observed outcome only after implementation and testing.]

CHAPTER 1 – INTRODUCTION

1.1 Background

University campuses contain classrooms, laboratories, libraries, cafeterias and common areas where students and staff may lose or find personal belongings. A centralized digital platform can help users report found items, search for missing belongings and coordinate verified returns.

1.2 Problem Statement

Lost-and-found information is often shared informally through notices, messaging groups or office visits. These methods make it difficult to search records, verify ownership, track claim status and maintain a transparent history. The project addresses the need for a centralized, searchable and role-based system.

1.3 Objectives

- Provide a single portal to report lost and found items.
- Allow users to search and filter reports by category, date and campus location.
- Enable claim submission and verification by authorized staff.
- Track item status from report creation to return or closure.
- Provide dashboards and reports for campus administrators.
- Deploy the application using containerization and a repeatable DevOps workflow.

1.4 Scope

The system covers item reporting, search, claim processing, staff verification, notifications, administrative management and reporting within a university campus. Payment processing and external identity-provider integration are outside the initial scope unless explicitly implemented.

1.5 Limitations and Assumptions

[List realistic constraints such as prototype data, email/SMS sandbox use, campus-only deployment, and assumptions.]

CHAPTER 2 – REQUIREMENT ANALYSIS

2.1 User Roles

Role	Responsibilities
Student / Staff	Register/login, report lost/found items, search listings, submit claims and track own requests.
Lost-and-Found Officer	Review reports, verify claim evidence, update item status and arrange handover.
Administrator	Manage users, categories, locations, access permissions and system reports.

2.2 Functional Requirements

ID	Requirement
FR-01	The system shall authenticate users and enforce role-based access.
FR-02	Users shall submit lost-item reports with description, category, date, location and optional image.
FR-03	Users shall submit found-item reports with custody/location details and optional image.
FR-04	Users shall search and filter items by category, date, location and status.
FR-05	Users shall submit a claim with ownership evidence without exposing sensitive claim details publicly.
FR-06	The officer shall approve, reject or request more information for a claim.
FR-07	The system shall track item and claim status with timestamps.
FR-08	The administrator shall manage users, categories, campus locations and reports.

2.3 Non-Functional Requirements

- Security: passwords must be hashed; authorization must be enforced on the server.
- Privacy: personal contact details and claim evidence must not be publicly exposed.
- Usability: interfaces should be responsive and accessible on desktop and mobile.
- Reliability: invalid operations should return clear errors and must not corrupt records.
- Performance: common searches should return results within an acceptable response time under the tested load.
- Maintainability: code should be modular, documented and version-controlled.

2.4 Minimum Screens

No.	Screen	Purpose
1	Login / Registration	Authenticate users and create accounts.

2	Role-based Dashboard	Show relevant counts, recent activity and shortcuts.
3	Report Lost Item	Create a lost-item report.
4	Report Found Item	Create a found-item report.
5	Browse and Search Items	Search/filter available reports.
6	Item Details	View permitted item information and submit a claim.
7	My Reports and Claims	Track user-submitted reports and claim status.
8	Officer Review Panel	Review reports, claims and evidence; update status.
9	Admin Management and Reports	Manage users/categories/locations and view/export reports.

2.5 Mandatory Reports

- Lost and Found Items Report – filter by date, category, location and status.
- Pending Claims Report – list claims awaiting officer action.
- Returned Items Report – list verified items successfully handed over.
- Category-wise and Location-wise Summary – show report volumes and trends.

CHAPTER 3 – SYSTEM DESIGN

3.1 Proposed Architecture

Use a three-tier architecture: (1) responsive frontend, (2) backend REST API and business services, and (3) relational database. Uploaded images should be stored in controlled file/object storage, with only metadata and secure references stored in the database. A reverse proxy may route client requests.

Layer	Suggested Technology (example)	Responsibility
Frontend	React / Angular / Vue	User interface, forms, validation and dashboards.
Backend	Node.js/Express / Java Spring Boot / Django	Authentication, APIs, business rules and authorization.
Database	MySQL / PostgreSQL	Users, items, claims, categories, locations and audit records.
File Storage	Local development storage or object storage	Item images and claim evidence with restricted access.
DevOps	GitHub, Docker, CI pipeline, Kubernetes	Version control, build, test, containerization and deployment.

3.2 Database Design

Entity / Table	Important Attributes (illustrative)
Users	user_id, name, email, password_hash, role, department, status
Categories	category_id, category_name, description
Locations	location_id, building, floor, area_name
Items	item_id, report_type, title, description, category_id, location_id, report_date, status, created_by
ItemImages	image_id, item_id, storage_key, uploaded_at
Claims	claim_id, item_id, claimant_id, claim_description, evidence_reference, status, reviewed_by, reviewed_at
HandoverRecords	handover_id, item_id, claim_id, handed_over_by, received_by, handover_time
AuditLogs	log_id, actor_id, action, entity_type, entity_id, timestamp

Figure 1. Architecture/Flow diagram

<<Give one paragraph explanation about the figure>>

CHAPTER 4 – IMPLEMENTATION

4.1 Technology Stack

Component	Selected Technology	Version / Notes
Frontend	[Enter]	[Enter]
Backend	[Enter]	[Enter]
Database	[Enter]	[Enter]
Authentication	[Enter]	[Enter]
Version Control	Git / GitHub	Repository URL: [Enter]
Containerization	Docker	Image/tag: [Enter]
Orchestration	Kubernetes	Cluster/environment: [Enter]

4.2 Module-wise Implementation

Module	Implementation Details / Evidence
Authentication and Roles	[Explain login, password hashing, sessions/tokens and server-side role checks.]
Lost/Found Reporting	[Explain forms, validation, image handling and database persistence.]
Search and Matching	[Explain filters, search logic and matching criteria.]
Claim Verification	[Explain evidence submission, officer review and status transitions.]
Administration and Reports	[Explain user/category management, dashboards and report export.]

4.3 API Documentation

Method	Endpoint	Purpose	Authorization
POST	/api/auth/login	Authenticate user	Public
POST	/api/items	Create lost/found report	Authenticated user
GET	/api/items	Search and list permitted items	Authenticated/public as designed
GET	/api/items/{id}	View item details	Role/ownership rules
POST	/api/items/{id}/claims	Submit claim	Authenticated user
PATCH	/api/claims/{id}/review	Review claim	Lost-and-Found Officer
GET	/api/reports/summary	View summary reports	Admin/Officer

[Update endpoint names, request/response examples, validation rules and error codes to match the actual implementation.]

Installation and Deployment Guide
API Request/Response Samples
Database Schema / SQL Scripts
GitHub Repository and CI/CD Evidence

Copy code here

CHAPTER 5 – TESTING AND RESULTS

5.1 Testing Strategy

Describe unit testing, API testing, integration testing, role/authorization testing, UI testing and basic performance testing.

5.2 Sample Test Cases

Test ID	Scenario	Expected Result	Actual Result	Status
TC-01	Login with valid credentials	User is authenticated and routed to correct dashboard	[Fill after test]	[Pass/Fail]
TC-02	Submit lost-item form with valid data	Report is saved and assigned a reference ID	[Fill after test]	[Pass/Fail]
TC-03	Submit report with missing required fields	Validation message is displayed; record is not created	[Fill after test]	[Pass/Fail]
TC-04	Student attempts officer-only claim review	Access is denied by backend authorization	[Fill after test]	[Pass/Fail]
TC-05	Officer approves a valid claim	Claim status and audit record are updated	[Fill after test]	[Pass/Fail]
TC-06	Search by category and location	Only matching records are displayed	[Fill after test]	[Pass/Fail]
TC-07	Generate pending claims report	Report includes claims awaiting review	[Fill after test]	[Pass/Fail]

5.3 Screenshots and Results

[Insert screenshots of each major screen and explain the result. Use actual screenshots from the running application; do not use mockups as implementation evidence.]

5.4 Defects and Fixes

Defect ID	Description	Root Cause	Fix Applied	Retest Result
[D-01]	[Enter]	[Enter]	[Enter]	[Pass/Fail]
[D-02]	[Enter]	[Enter]	[Enter]	[Pass/Fail]

CHAPTER 6 – DEVOPS AND DEPLOYMENT

6.1 Git and GitHub Workflow

Provide repository URL, branch strategy, commit evidence, pull-request/review approach and release tags.

6.2 Continuous Integration / Continuous Deployment

Describe the pipeline stages actually implemented: checkout, dependency installation, linting, unit tests, build, container image creation and deployment. Include a pipeline screenshot and execution result.

6.3 Docker Implementation

Include the Dockerfile, image build command, container run command, environment configuration and successful execution evidence.

6.4 Kubernetes Deployment

Include Deployment and Service YAML files, replica configuration, ports, environment variables/secrets handling, deployment commands and pod/service status screenshots.

6.5 Monitoring and Logging

Describe application logs, health checks, error monitoring and any dashboard/alerts implemented.

CHAPTER 7 – CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

[Summarize implemented features, tested workflows and deployment outcomes. Avoid claiming features that were not completed.]

7.2 Future Enhancements

- Campus SSO integration
- Email/SMS notifications
- Image similarity-based item matching
- QR-coded handover receipt
- Mobile application

REFERENCES

[List official documentation, textbooks, research papers, APIs and libraries used. Use a consistent citation style.]